
```

rm(list = ls())

# Local github
codepath <- c("/media/larryleon/My Projects/GitHub/Forest-Search/R/")
source(paste0(codepath, "source_forestsearch_v0.R"))
source_fs_functions(file_loc = codepath)

library(kableExtra)
library(knitr)
library(ggplot2)
library(gridExtra)
library(cubature)
library(aVirtualTwins)
library(randomForest)
library(survival)
library(survminer)
library(grf)
library(policytree)
library(data.table)
library(plyr)
library(dplyr)
library(glmnet)
library(corrplot)

maxFollow <- 84
cens.type <- "weibull"

# m1 -censoring adjustment
muC.adj <- log(1.5)
k.z3 <- 1
k.treat <- 0.9

z1_frac <- 0.25 # Default model index 'm1' (The 1st quartile of z1=er)
pH_super <- 0.125 # non-NULL re-defines z1_frac

if (is.null(pH_super)) {
  # pH_check<-with(gbbsg,mean(pgr<=quantile(pgr,c(z3_frac),1,0) &
  # er<=quantile(er,z1_frac)))
  pH_check <- with(gbbsg, mean(meno == 0 & er <= quantile(er, z1_frac)))
  cat("Underlying pH_super", c(pH_check), "\n")
}
# pH_super specified If pH_super then override z1_frac and find z1_frac to
# yield pH_super

if (!is.null(pH_super)) {
  # Approximate Z1 quantile to yield pH proportion
  z1_q <- uniroot(propH.obj4, c(0, 1), tol = 1e-04, pH.target = pH_super)$root
  # pH_check<-with(gbbsg,mean(pgr<=quantile(pgr,c(z3_frac),1,0) &
  # er<=quantile(er,z1_q)))
  pH_check <- with(gbbsg, mean(meno == 0 & er <= quantile(er, z1_q)))
  cat("pH", c(pH_check), "\n")
  rel_error <- (pH_super - pH_check)/pH_super
  if (abs(rel_error) >= 0.1)
    stop("pH_super approximation relative error exceeds 10%")
  z1_frac <- z1_q
}

```

```

    cat("Underlying pH_super", c(pH_check), "\n")
  }

## pH 0.122449
## Underlying pH_super 0.122449

# Bootstrap on log(hr) scale converted to HR (est.loghr=TRUE & est.scale='hr')
t.start.all <- proc.time()[3]

##### Forest search criteria
hr.threshold <- 1.25 # Initial candidates
hr.consistency <- 1 # Candidates for many splits
pconsistency.threshold <- 0.9
stop.threshold <- 0.95
maxk <- 2
nmin.fs <- 60
pstop_futile <- 0.5
# Limit timing for forestsearch
max.minutes <- 3
m1.threshold <- Inf # Turning this off (Default)
# pconsistency.threshold<-0.70 # Minimum threshold (will choose max among
# subgroups satisfying)
fs.splits <- 400 # How many times to split for consistency
# vi is % factor is selected in cross-validation --> higher more important
vi.grf.min <- 0.2
# Null, turns off grf screening
d.min <- 10 # Min number of events for both arms (d0.min=d1.min=d.min)
# default=5 Virtual twins analysis Counter-factual difference (C-E) >=
# vt.threshold Large values in favor of C (control)
vt.threshold <- 0.225 # For VT delta
treat.threshold <- 0

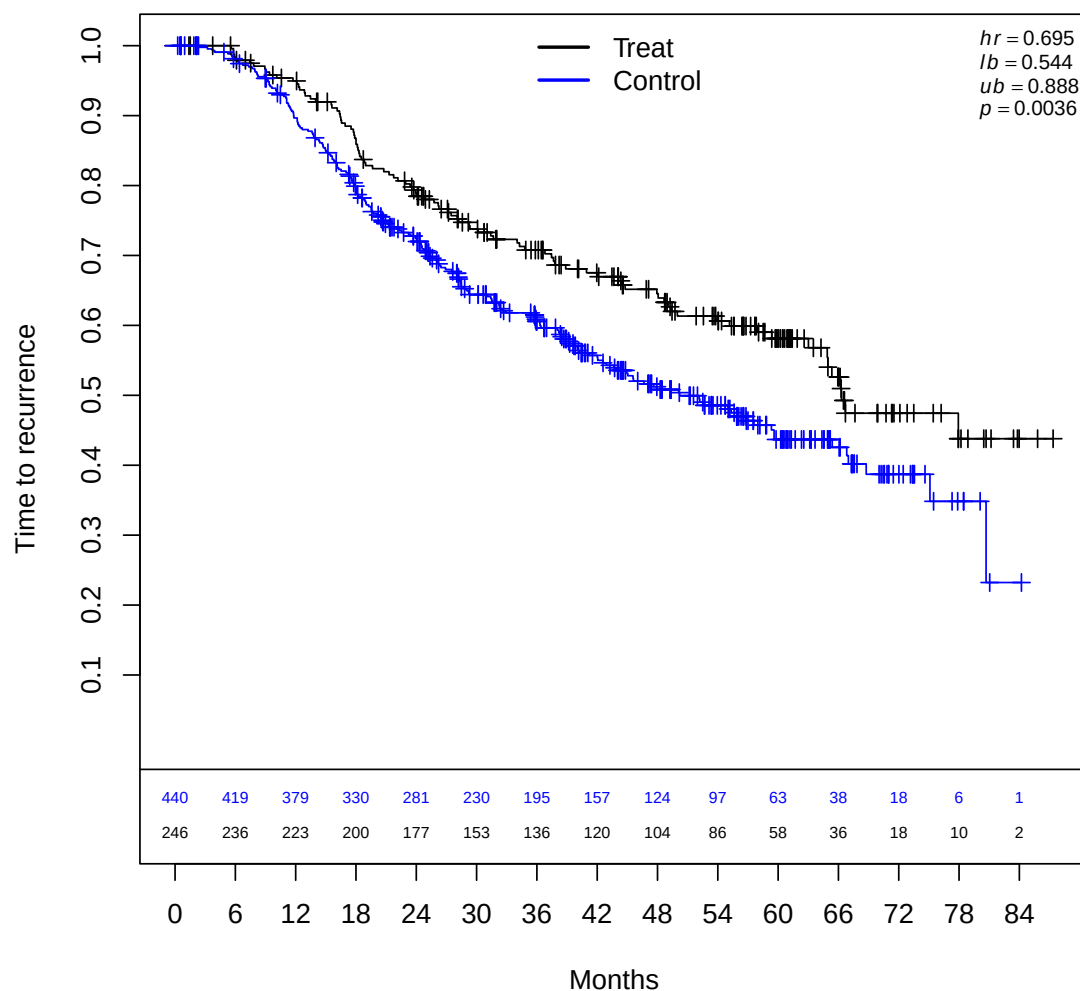
maxdepth <- 2
n.min <- 60
ntree <- 1000
# GRF criteria
dmin.grf <- 12 # For GRF delta
# Note: For CRT this represents dmin.grf/2 RMS for control (-dmin.grf/2 for
# treatment)
frac.tau <- 0.6

outcome.name <- c("y.sim")
event.name <- c("event.sim")
id.name <- c("id")
treat.name <- c("treat")

cox.formula.sim <- as.formula(paste("Surv(y.sim,event.sim)~treat"))
cox.formula.adj.sim <- as.formula(paste("Surv(y.sim,event.sim)~treat+v1+v2+v3+v4+v5"))

mod.harm <- "alt"
hrH.target <- 2
# out.loc = NULL turns off file creation
N <- 1000
this.dgm <- get.dgm4(mod.harm = mod.harm, N = N, k.treat = k.treat, hrH.target = hrH.target,
  cens.type = cens.type, out.loc = NULL, details = TRUE, parms_torand = FALSE)

```



```
## Super-population empirical harm and non-harm hazard ratios= 2.000007 0.6466405
## Causal HR (empirical ITT)= 0.7057463
```

```
# sim sensH ppvH sensHc ppvHc 1: 11 0.5000000 0.5731707 0.9422442 0.9239482 2:
# 14 0.6891892 0.5666667 0.9376997 0.9622951 3: 17 0.3296703 0.2830189
# 0.8752053 0.8973064 4: 18 0.4489796 0.4943820 0.9252492 0.9116203 5: 20
# 0.8045977 0.4347826 0.8515498 0.9684601 6: 23 0.8101266 0.5203252 0.9049919
# 0.9740035 7: 30 0.7802198 0.5182482 0.8916256 0.9644760

file_out <- c("output/sim-FS4_sim10.Rdata")

n_add_noise <- 3

sim <- 10

dgm <- this.dgm$dgm

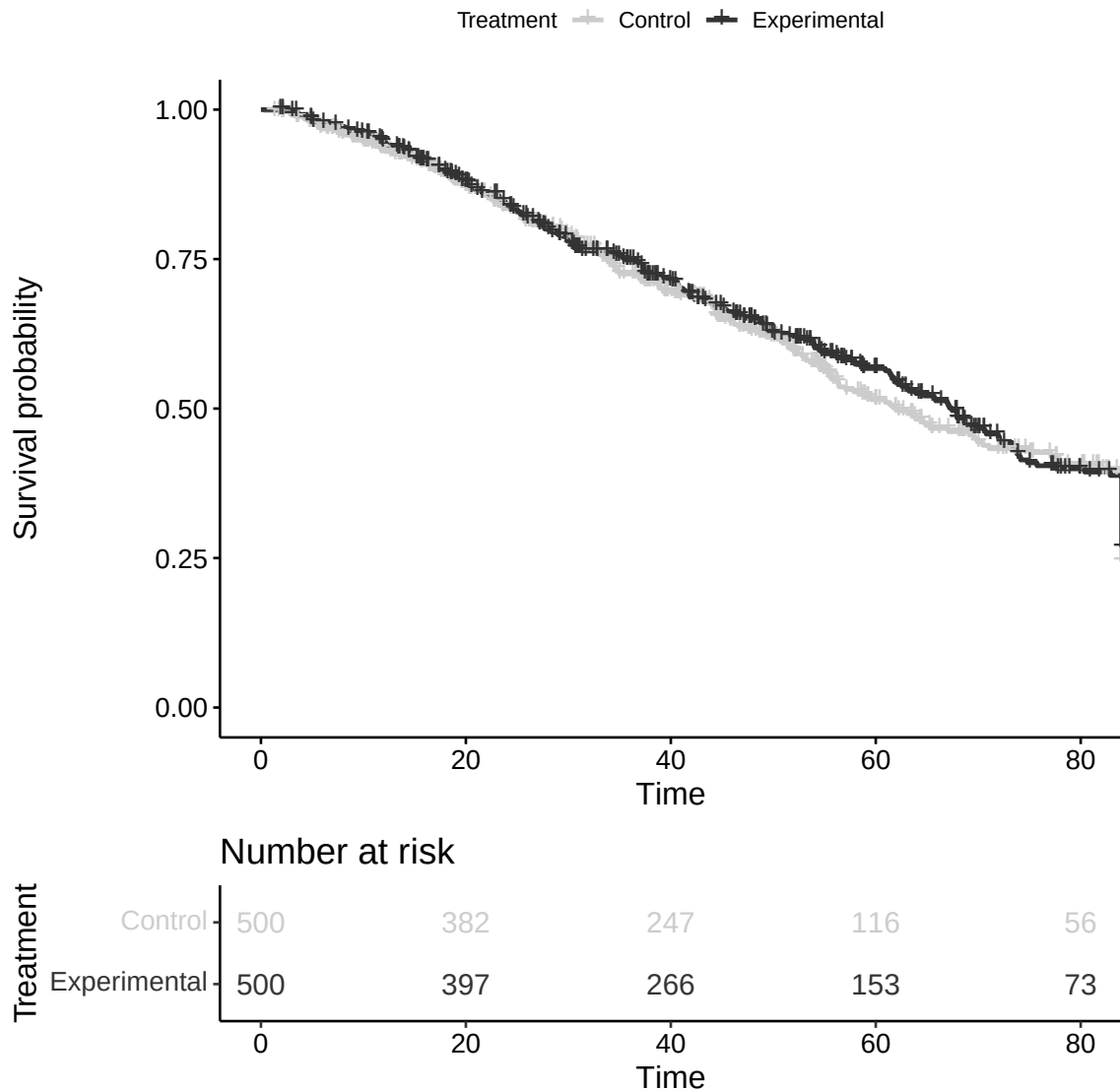
x <- sim_aftm4_gbsg(dgm = dgm, n = N, maxFollow = maxFollow, muC.adj = muC.adj, simid = sim)
```

```

kmfit <- survfit(Surv(y.sim, event) ~ treat, data = x)

ggsurvplot(kmfit, data = x, main = "K-M curves for simulated data", legend = "top",
  legend.title = "Treatment", legend.labs = c("Control", "Experimental"), palette = "grey",
  risk.table = TRUE, risk.table.col = "strata")

```



```

if (n_add_noise == 0) {
  confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade3")
}

if (n_add_noise == 5) {
  set.seed(8316951 + 1000 * sim)
  # Add 5 noise
  x$noise1 <- rnorm(N)
  x$noise2 <- rnorm(N)
  x$noise3 <- rnorm(N)
  x$noise4 <- rnorm(N)
  x$noise5 <- rnorm(N)
}

```

```

    confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade3", "noise1",
                          "noise2", "noise3", "noise4", "noise5")
  }

  if (n_add_noise == 3) {
    set.seed(8316951 + 1000 * sim)
    # Add 3 noise
    x$noise1 <- rnorm(N, sd = 1)
    x$noise2 <- rnorm(N, sd = 1)
    x$noise3 <- rnorm(N, sd = 1)
    confounders.name <- c("z1", "z2", "z3", "z4", "z5", "size", "grade3", "noise1",
                          "noise2", "noise3")
  }

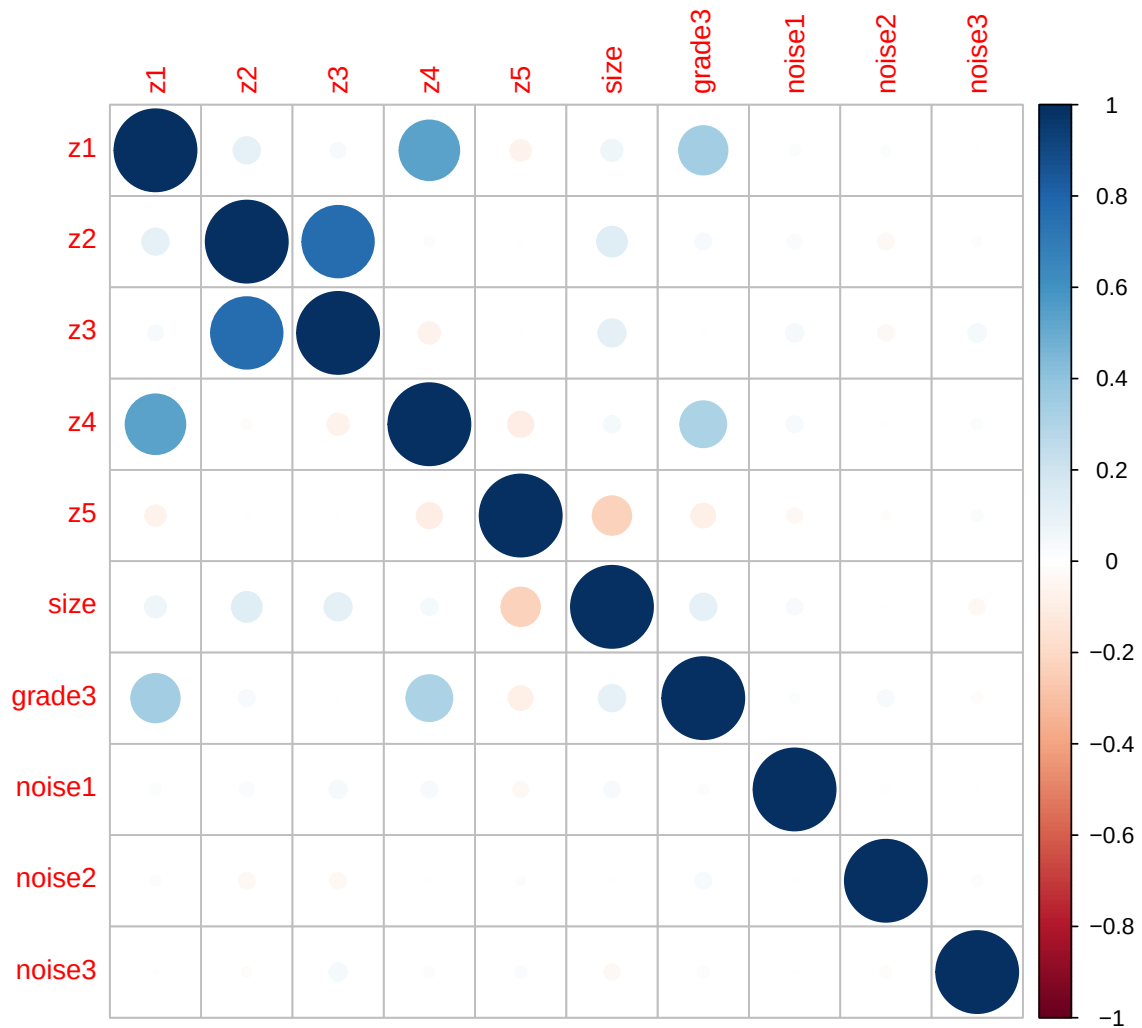
  # More challenging which_replace <- which(confounders.name=='z1')
  # confounders.name[which_replace] <- 'er'

  cox.formula.check <- as.formula(paste("Surv(y.sim,event.sim)~treat+v1+v2+v3+v4+v5+size+grade3+noise1",
                                       "noise2+noise3"))
  coxph(cox.formula.check, data = x)

## Call:
## coxph(formula = cox.formula.check, data = x)
##
##              coef exp(coef) se(coef)      z      p
## treat  -0.308852  0.734290  0.085521  -3.611 0.000305
## v11     0.552833  1.738170  0.104292   5.301 1.15e-07
## v21    -0.151950  0.859031  0.135396  -1.122 0.261749
## v31     0.082830  1.086357  0.138097   0.600 0.548642
## v41     0.480078  1.616201  0.100935   4.756 1.97e-06
## v51    -1.031314  0.356538  0.091077 -11.324 < 2e-16
## size   -0.003580  0.996426  0.003087  -1.160 0.246081
## grade3 -0.020456  0.979752  0.104577  -0.196 0.844919
## noise1 -0.004180  0.995829  0.041281  -0.101 0.919349
## noise2  0.059344  1.061140  0.042891   1.384 0.166479
## noise3 -0.021087  0.979133  0.043310  -0.487 0.626333
##
## Likelihood ratio test=254.7 on 11 df, p=< 2.2e-16
## n= 1000, number of events= 566

Zm <- cor(as.matrix(x[, c(confounders.name)]))
corrplot(Zm)

```



```
# Options Allconfounders.name is list of confounders within analysis dataset
# (1) use_lasso=TRUE & use_grf=FALSE Lasso used to possibly reduce dimension
# Any continuous factors are cut at medians (2) use_lasso=TRUE & use_grf=TRUE
# Lasso used to reduce dimension Continuous covariates are cut at medians
# However, if GRF selects a covariate cut then only that cut is used: For
# example if 'age <= median(ag)' is called for per Lasso but GRF includes 'age
# <= 54', then only the latter is used (3) use_grf_only = TRUE (overrides
# use_lasso and use_grf) Only factors selected via GRF are used (4) use_lasso =
# F & use_grf =T Median cuts (unless selected via GRF) as in (2) However no
# possible dimension reduction via lasso All categorical factors included

use_lasso <- TRUE
use_grf <- TRUE
use_grf_only <- FALSE

fs.est <- forestsearch(df.analysis = x, Allconfounders.name = confounders.name, details = TRUE,
  use_lasso = use_lasso, use_grf = use_grf, use_grf_only = use_grf_only, dmin.grf = 12,
  frac.tau = 0.6, conf_force = NULL, outcome.name = outcome.name, treat.name = treat.name,
```

```

event.name = event.name, id.name = id.name, n.min = nmin.fs, hr.threshold = hr.threshold,
hr.consistency = hr.consistency, fs.splits = fs.splits, d0.min = d.min, d1.min = d.min,
pstop_futile = pstop_futile, pconsistency.threshold = pconsistency.threshold,
stop.threshold = stop.threshold, max.minutes = max.minutes, maxk = maxk, by.risk = 12,
plot.sg = TRUE, vi.grf.min = vi.grf.min)

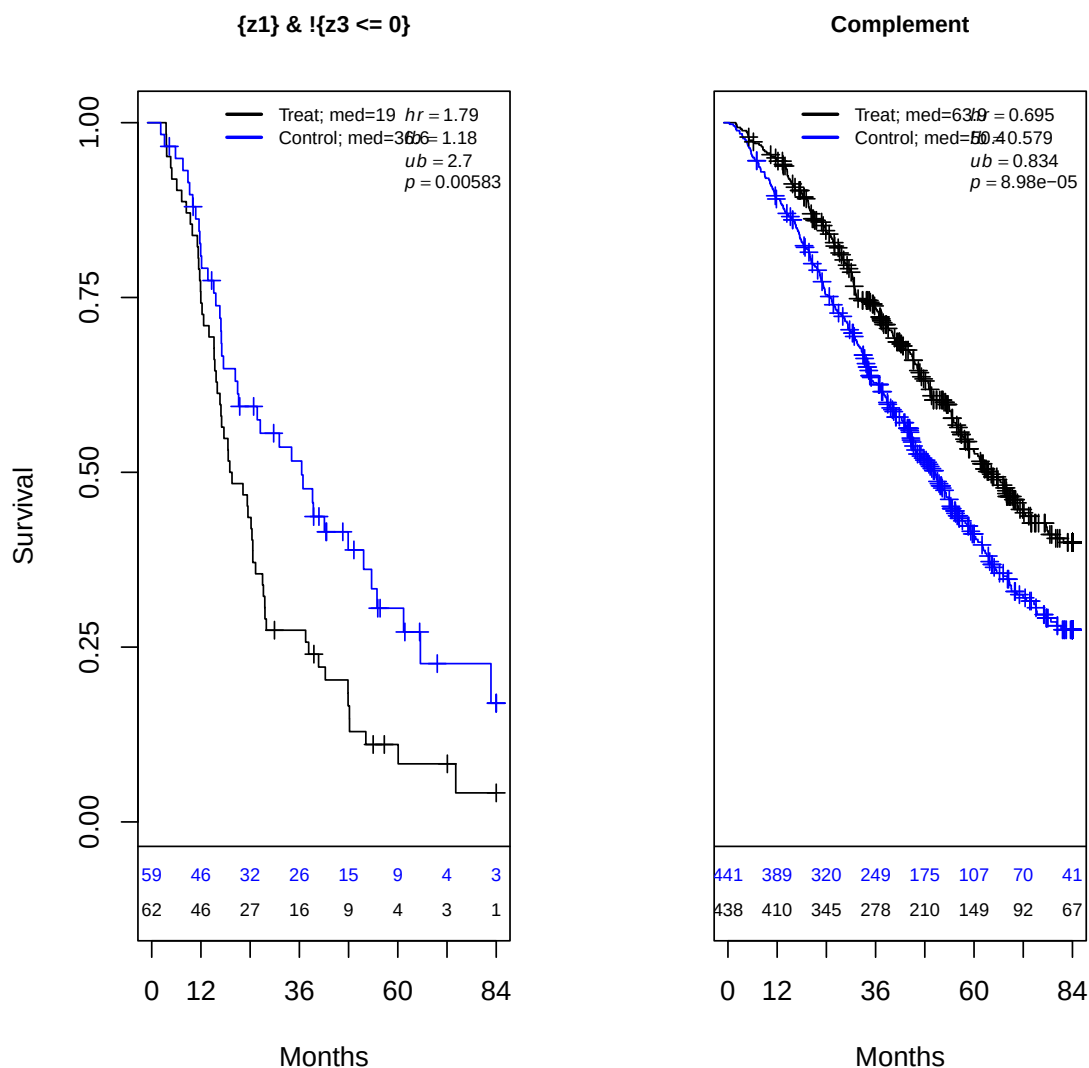
## tau, maxdepth= 49.0844 2
##   leaf.node control.mean control.size control.se treated.mean treated.size
## 1         2  -4.6161019  605.0000000   1.1526452   4.6161019  605.0000000
## 2         3   0.6533015  395.0000000   1.3952607  -0.6533015  395.0000000
## 21        5  -3.9198496  706.0000000   1.0101819   3.9198496  706.0000000
## 3         6  -5.5649660  151.0000000   2.4297894   5.5649660  151.0000000
## 4         7   7.8911227  121.0000000   2.8728716  -7.8911227  121.0000000
##   treated.se      diff Nsg depth
## 1   1.1526452  -9.232204 605     1
## 2   1.3952607   1.306603 395     1
## 21  1.0101819  -7.839699 706     2
## 3   2.4297894 -11.129932 151     2
## 4   2.8728716  15.782245 121     2
##   leaf.node control.mean control.size control.se treated.mean treated.size
## 4         7   7.891123   121.000000   2.872872  -7.891123   121.000000
##   treated.se      diff Nsg depth
## 4   2.872872 15.78225 121     2
## [1] "z1 <= 0"           "noise3 <= -1.99" "z3 <= 0"
## [1] "z3 <= 0"
## # of continuous/categorical characteristics 4 6
## Continuous characteristics: size noise1 noise2 noise3
## Categorical characteristics: z1 z2 z3 z4 z5 grade3
## 10 x 1 sparse Matrix of class "dgCMatrix"
##               s0
## z1      0.4846674988
## z2     -0.0009472776
## z3      .
## z4      0.4008096864
## z5     -0.9241174615
## size      .
## grade3     .
## noise1     .
## noise2  0.0178622886
## noise3     .
## Cox-LASSO selected: z1 z2 z4 z5 noise2
## Cox-LASSO not selected: z3 size grade3 noise1 noise3
## Median cuts (after Lasso if use_lasso) noise2
## Categorical (after Lasso if use_lasso) z1 z2 z4 z5
## Prior to removing (if in GRF): noise2
## Left (to cut at median) after removing others if in GRF: noise2
## Initial GRF cuts included z1 <= 0 noise3 <= -1.99 z3 <= 0
## # of candidate subgroup factors= 7
## [1] "noise2 <= median(noise2)" "noise3 <= -1.99"
## [3] "z1"                      "z2"
## [5] "z4"                      "z5"
## [7] "z3 <= 0"
## Confounders per grf screening q3 q7 q4 q6 q5 q1
##   FSconfounders.name      vi.cs      vi_ratio
## 3                q3 2.791033e-01 1.9537234047
## 7                q7 2.160418e-01 1.5122926764

```

```

## 4          q4 1.510066e-01 1.0570459882
## 6          q6 1.185037e-01 0.8295262186
## 5          q5 1.182749e-01 0.8279246124
## 1          q1 1.169811e-01 0.8188679769
## 2          q2 8.844611e-05 0.0006191228
## Number of unique levels (L) and possible subgroups= 12 4095
## # of subgroups based on # variables > k.max and excluded (per million) 0.004017
## k.max= 2
## Events criteria for control,exp= 10 10
## # of subgroups with events less than criteria: control, experimental 8 8
## # of subgroups with sample size less than criteria 8
## # of subgroups meeting all criteria = 70
## # of subgroups fitted (Cox model estimable) = 70
## *Subgroup Searching Minutes=* 0.003383333
## Number of subgroups meeting HR threshold 3
## # of candidate subgroups (meeting HR criteria) = 3
## Subgroups (1st 10) meeting overall screening thresholds (HR, m1) sorted by HRs= Inf
##      K    n    E d1    m1    m0    HR L(HR) U(HR) q3.0 q3.1 q7.0 q7.1 q4.0 q4.1
## 1:  2 121  95 56 19.31 36.56 1.79  1.18  2.70    0    1    1    0    0    0
## 2:  2 168 125 72 25.32 36.88 1.31  0.92  1.88    0    1    0    0    0    1
## 3:  2 147 128 75 21.59 26.17 1.26  0.88  1.80    0    1    0    0    0    0
## 4: NA  NA  NA NA    NA    NA    NA    NA    NA    NA    NA    NA    NA    NA
## 5: NA  NA  NA NA    NA    NA    NA    NA    NA    NA    NA    NA    NA    NA
## 6: NA  NA  NA NA    NA    NA    NA    NA    NA    NA    NA    NA    NA    NA
## 7: NA  NA  NA NA    NA    NA    NA    NA    NA    NA    NA    NA    NA    NA
## 8: NA  NA  NA NA    NA    NA    NA    NA    NA    NA    NA    NA    NA    NA
## 9: NA  NA  NA NA    NA    NA    NA    NA    NA    NA    NA    NA    NA    NA
## 10: NA NA  NA NA    NA    NA    NA    NA    NA    NA    NA    NA    NA    NA
##      q6.0 q6.1 q5.0 q5.1 q1.0 q1.1
## 1:    0    0    0    0    0    0
## 2:    0    0    0    0    0    0
## 3:    1    0    0    0    0    0
## 4:   NA   NA   NA   NA   NA   NA
## 5:   NA   NA   NA   NA   NA   NA
## 6:   NA   NA   NA   NA   NA   NA
## 7:   NA   NA   NA   NA   NA   NA
## 8:   NA   NA   NA   NA   NA   NA
## 9:   NA   NA   NA   NA   NA   NA
## 10:  NA   NA   NA   NA   NA   NA
## Consistency 0.9925
## # of splits= 400
## Model, % Consistency Met= {z1} !{z3 <= 0} 0.9925
## Number of subgroups meeting consistency criteria=
##      p.consistency Nsg group.id m.index K    M.1          M.2
## 1:      0.9925 121          1      1 2 {z1} !{z3 <= 0}

```



```
## [1] "{z1}"      "!("{z3 <= 0}"
## % consistency criteria met= 0.9925
## SG focus= hr
## Subgroup Consistency Minutes= 0.02355
## Subgroup found
## Minutes overall= 0.05818333

grf.est <- grf.subg.harm.survival(data = x, confounders.name = confounders.name,
  outcome.name = outcome.name, event.name = event.name, id.name = id.name, treat.name = treat.name,
  n.min = n.min, dmin.grf = dmin.grf, frac.tau = 1, details = TRUE)

## tau, maxdepth= 81.80734 2
##   leaf.node control.mean control.size control.se treated.mean treated.size
## 1         2    -8.273341   728.000000   1.878638     8.273341   728.000000
## 2         3     1.286521   272.000000   3.039867    -1.286521   272.000000
## 21        5    -10.850378   563.000000   2.138454    10.850378   563.000000
## 3         6    -5.928266   296.000000   2.858962     5.928266   296.000000
## 4         7    15.111153   121.000000   4.445754    -15.111153   121.000000
##   treated.se      diff Nsg depth
```

```
## 1      1.878638 -16.546683 728      1
## 2      3.039867   2.573043 272      1
## 21     2.138454 -21.700757 563      2
## 3      2.858962 -11.856531 296      2
## 4      4.445754  30.222306 121      2
##      leaf.node control.mean control.size control.se treated.mean treated.size
## 4           7      15.111153   121.000000   4.445754  -15.111153   121.000000
##      treated.se      diff Nsg depth
## 4      4.445754 30.22231 121      2
## [1] "z3 <= 0"           "noise3 <= -1.99" "z1 <= 0"
## [1] "z1 <= 0"
```

```
library(doParallel)
registerDoParallel(parallel::detectCores(logical = FALSE))

cox.formula.boot <- cox.formula.sim
max.minutes <- 7
# Suggest running 50, first ... to get timing estimate

NB <- 1000

df_boot_analysis <- fs.est$df.est

fitH <- get_Cox_sg(df_sg = subset(df_boot_analysis, treat.recommend == 0), cox.formula = cox.formula)
H_obs <- fitH$est_obs # log(hr) scale
seH_obs <- fitH$se_obs
# Hc observed estimates
fitHc <- get_Cox_sg(df_sg = subset(df_boot_analysis, treat.recommend == 1), cox.formula = cox.formula)
Hc_obs <- fitHc$est_obs
seHc_obs <- fitHc$se_obs
rm("fitH", "fitHc")

Ystar_mat <- bootYstar({
  ystar <- get_Ystar(boot)
}, boots = NB, seed = 8316951, counter = "boot", export = fun_arg_list_boot)
# Check dimension
if (dim(Ystar_mat)[1] != NB | dim(Ystar_mat)[2] != nrow(df_boot_analysis)) stop("Dimension of Ystar_")

tB.start <- proc.time()[3]
# Bootstraps
resB <- bootPar({
  ans <- fsboot_forparallel(boot)
}, boots = NB, seed = 8316951, counter = "boot", export = fun_arg_list_boot)
tB.now <- proc.time()[3]
tB.min <- (tB.now - tB.start)/60

doParallel::stopImplicitCluster()

cat("Minutes for Boots", c(NB, tB.min), "\n")
## Minutes for Boots 1000 3.775067

cat("Projection per 1000", c(tB.min * (1000/NB)), "\n")
## Projection per 1000 3.775067

cat("Propn bootstrap subgroups found =", c(sum(!is.na(resB$H_biasadj_1))/NB), "\n")
```

```

## Propn bootstrap subgroups found = 0.866

# How many timed out
cat("Number timed out=", c(sum(is.na(resB$H_biasadj_1) & resB$tmins_search > max.minutes)),
    "\n")

## Number timed out= 0

H_estimates <- get_dfRes(Hobs = H_obs, seHobs = seH_obs, H1_adj = resB$H_biasadj_2,
    ystar = Ystar_mat, cov_method = "standard", cov_trim = 0)
Hc_estimates <- get_dfRes(Hobs = Hc_obs, seHobs = seHc_obs, H1_adj = resB$Hc_biasadj_2,
    ystar = Ystar_mat, cov_method = "standard", cov_trim = 0)

print(H_estimates)

##           H0           sdH0 H0_lower H0_upper           H1           sdH1 H1_lower H1_upper
## 1: 1.787969 0.3768169 1.182955 2.702414 1.486584 0.3401396 0.9493567 2.327821

print(Hc_estimates)

##           H0           sdH0 H0_lower H0_upper           H1           sdH1 H1_lower
## 1: 0.6949098 0.06457867 0.5791958 0.8337416 0.7081791 0.09500922 0.544435
##           H1_upper
## 1: 0.9211709

# H_estimates<-get_dfRes(Hobs=H_obs,seHobs=seH_obs,H1_adj=resB$H_biasadj_1,ystar=Ystar_mat,cov_metho
# Hc_estimates<-get_dfRes(Hobs=Hc_obs,seHobs=seHc_obs,H1_adj=resB$Hc_biasadj_1,ystar=Ystar_mat,cov_m
# print(H_estimates) print(Hc_estimates)

bootit <- list(H_estimates = H_estimates, Hc_estimates = Hc_estimates)
ans3 <- fsBoot.parallel.out(df = df_boot_analysis, FSboots = bootit, cox.formula.boot = cox.formula.
    sim_number = sim)
tall.min <- (tB.now - t.start.all)/60
cat("Overall minutes for analysis", c(tall.min), "\n")

## Overall minutes for analysis 3.918

save(dgm, x, fs.est, grf.est, bootit, ans3, tall.min, resB, cox.formula.boot, file = file_out)

# ITT causal hazard ratio mean(fs.est$df.est$hlin.ratio) H Oracle dfH <-
# subset(fs.est$df.est, flag.harm==1) mean(dfH$hlin.ratio) dfH_est <-
# subset(fs.est$df.est, treat.recommend==0) mean(dfH_est$hlin.ratio) dfHc <-
# subset(fs.est$df.est, flag.harm==0) mean(dfHc$hlin.ratio) dfHc_est <-
# subset(fs.est$df.est, treat.recommend==1) mean(dfHc_est$hlin.ratio) dfSens <-
# subset(fs.est$df.est, flag.harm==1 & treat.recommend==0) sensH <-
# nrow(dfSens)/nrow(dfH)

## Confounders evaluated in Forestsearch:
## [1] "noise2 <= median(noise2)" "noise3 <= -1.99"
## [3] "z1" "z2"
## [5] "z4" "z5"
## [7] "z3 <= 0"

```

Table 1 Simulated data example: Cox hazard ratio (hr) estimates for the ITT population and subgroups H and H^c . Cox model estimates are based on subgroups: H true (knowing the actual subgroup, a-priori); the estimated subgroup $\hat{H}$; and the bootstrap (1,000 resamples) bias-correction to $\hat{H}$ estimates, denoted $\hat{H}_{bc}$. Estimates for the complement H^c are defined analogously. The number of subjects in each population (# Subjects) and the number of subjects correctly classified (# Correct) in the true subgroups H and H^c , respectively, are listed.

	HR Estimate	Lower	Upper	# Subjects	# Correct
ITT	0.795	0.674	0.938	1000	.
H subgroup estimates					
H_{oracle}	1.788	1.183	2.703	121	.
$\hat{H}$	1.788	1.183	2.703	121	121
$\hat{H}_{bc}$	1.487	0.95	2.328	121	121
H-complement subgroup estimates					
H_{oracle}^c	0.695	0.579	0.835	879	.
$\hat{H}^c$	0.695	0.579	0.835	879	879
$\hat{H}_{bc}^c$	0.708	0.544	0.921	879	879